

Files

Overview

- Text files
 - File Writing
 - File Reading
- CSV

Why files?

File reading and writing

Long-term Storage

- So far, we have learned how to store and manipulate data using variables.

How long does a variable exist?

The Need For Long-term Storage

- **Computer memory is volatile**
 - RAM, cache, registers
 - Power OFF, it is all gone
 - Python program finishes, it is all gone
- **Computers use “disk” for longer term storage**
 - A physical hard disk that mechanically spins
 - Flash drives without mechanical parts
 - “The cloud” (sends it off to a physical disk)

Python can read from and write to files

Reading From Files – 1

Reading data in from a (local) file using Python:

- `read()` reads the entire file
- `readlines()` reads into a list of lines
- `for` iterates over lines in the file

```
FILENAME = 'jabberwocky.txt'

# open the file and read the entire
contents
text = open(FILENAME).read()

# open the file and read contents into
list of lists
lines = open(FILENAME).readlines()
fp = open(FILENAME) \\ a "file pointer"
for line in fp:
    ...
fp.close()
```

Reading From Files – 2

Task: Given a file of tram stops and their routes, print a list of routes for each stop.

- Each line of the file contains two values, the stop and the route.
- Example line from file: Melbourne Central,38

#this means route 38 stops at Melbourne Central

- Hint: split() method and rstrip() methods may be useful
- (At least 10 minutes work here)

tram_routes.txt

```
Fed Square,69
Fed Square,78
Tennis Centre,3
Tennis Centre,38
Aquarium,38
Docklands,3
Melbourne-Central,38
Melbourne-Central,3
```

Write To Files

Writing to a file

```
FILENAME = 'file.txt'  
text = open(FILENAME , 'w')  
text.write('Tim woz ere')  
text.close ()
```

- Opening a file for writing, add parameter argument **'w'**. This overwrites any existing content

Append to a file

```
FILENAME = 'file.txt'  
text = open(FILENAME , 'a')  
text.write('Tim woz ere again')  
text.close ()
```

- Opening a file for appending, add parameter argument **'a'**. This adds new data to the end of any existing content.

The With Statement

- A ***with statement*** can be used to open a file, execute a block of statements, and automatically close the file when complete.

```
FILENAME = 'file.txt'

# opens the file to append and closes at end of loop
with open(FILENAME, 'a') as fp:
    fp.write('Tim woz ere')

# reads from the file line by line and closes at end
with open(FILENAME, 'r') as fp:
    for line in fp.readlines():
        print(line)
```

Other files?

CSV Files

What Is A CSV File?

- A common data format is CSV (“comma-separated values”) files store tabulated data (think spreadsheets) as plain-text, separated by commas:
 - Year ,Make ,Model
 - 1997 ,Ford ,Falcon
 - 2006 , Honda ,Odyssey
- So common, in fact, that there is a csv module that helps you read them

Reading Data From CSV Files

- We can parse CSV files using `csv.reader` from the `csv` library, which imports the data into a list of lists:

```
import csv

FNAME = "rainfall.csv"
with line in csv.reader(open(FNAME)):
    print(line)
```

- Sample output
 - ['Station', 'Year', 'Month', 'Day', 'Rain_mm']
 - ['86035', '2019', '1', '3', '0']

Writing to CSV Files

```
import csv

row1 = ['Station', 'Year', 'Month', 'Day', 'Rain_mm']
row2 = ['86035', '2019', '1', '3', '0']

with open('rainfall.csv', 'w', newline='') as csvfile:
    grades_writer = csv.writer(csvfile)
    grades_writer.writerow(row1)
    grades_writer.writerow(row2)
    grades_writer.writerows([row1, row2])
```

Using The Data From DictReader

- As **DictReader** returns each row as an Ordered dictionary you can access data from a row by the column headings.

```
import csv
```

```
FNAME = 'rainfall.csv'  
def max_rainfall(data):  
    max_rain = None  
    for row in data:  
        rain = float(row["Rain_mm"])  
        if max_rain == None or max_rain < rain:  
            max_rain = rain  
    return max_rain
```

```
data = csv.DictReader(open(FNAME))  
rain = max_rainfall(data)  
if rain != None:  
    print(f"The maximum rainfall from data was {rain}mm.")  
else:  
    print("The file does not contain any rainfall data.")
```

First row column headings
['Station', 'Year', 'Month', 'Day', 'Rain_mm']
become the keys for each row of data in that column

Open file using csv.DictReader

Coding To Minimize Errors

Good Code Style And Handling Errors

Goof Programming Practices

- Using modular programming techniques (functions for each sub-task)
- Coding and testing one task at a time
- Tracing the execution (by hand and using Python tutor)
- Commenting your code and giving descriptive names to variables
- **New Technique:**
 - “Catching” or handling the exception

When Errors (Exceptions) Occur

We know Python prints out the Exception that cased the error:

```
>>> 9/0
Traceback (most recent call last):
File "<web session >", line 1, in <module >
ZeroDivisionError: integer division or modulo by zero

>>> 1 + "2"
Traceback (most recent call last):
File "<web session >", line 1, in <module >
TypeError: unsupported operand type(s) for +: 'int' and 'str'

>>> 1 + i
Traceback (most recent call last):
File "<web session >", line 1, in <module >
NameError: name 'i' is not defined
```

Exception Handling

- It is possible to “handle” exceptions within your code using:

```
try:  
    code statement(s) ...  
except Exception:  
    code run when exception occurs...
```

- **try** attempts to execute its block of code and passes off to the **exception** handlers (which are also tested in linear order) only if an exception is raised during the execution.

Exception Handling: An Example

```
while True:
    try:
        x = int (input("Enter a number: "))
        break
    except ValueError:
        print ("Oops ! Try again ...")
```

TASK: Reading / Writing A .txt File

- Can we read a file and write back to it or another file from the one program?
 - Yes. Imagine a simple command line menu something like the one below.

```
***** Main menu *****  
1.....add new data to file  
2.....display data  
3.....exit program  
Enter a menu option ——>
```

Do it on
ZyBooks

How?

- Simply incorporate the relevant read & write functions into the one program.
- Separate read and write tasks should be created in your program

Summary

- **Today we covered:**
 - Reading from files
 - Writing to files
 - Using the csv reader commands for csv files
- **Extension concepts:**
 - Using the csv DictReader commands for csv files
 - `try: ... except:` to catch and handle errors

Thank you
